6.1 노드 아키텍처 분석

개요

Composable Finance 프로젝트의 노드 아키텍처는 Substrate 프레임워크를 기반으로 한 파라체인 노드로 구성되어 있습니다. 이 문서에서는 `service.rs` 의 노드 서비스 초기화 과정, 클라이언트 구성 요소, 그리고 파라체인 콜레이터와 릴레이체인 연동 메커니즘을 상세히 분석합니다.

6.1.1 노드 서비스 초기화 과정

메인 서비스 초기화 함수

노드의 핵심 초기화는 `start_node_impl` 함수에서 이루어집니다:

```
async fn start_node_impl<RuntimeApi, Executor>(
parachain_config: Configuration,
polkadot_config: Configuration,
collator_options: CollatorOptions,
para_id: ParaId,
dalek_end_block: Option<u32>,
) -> sc_service::error::Result<TaskManager>
```

이 함수는 다음과 같은 주요 단계를 거쳐 노드를 초기화합니다:

1. **설정 준비**: `prepare_node_config` 를 통해 파라체인 설정을 준비
2. **부분 컴포넌트 생성**: `new_partial` 함수로 기본 컴포넌트들을 생성

3. **릴레이체인 인터페이스 구축**: `build_relay_chain_interface` 로 릴레이체인과의 연결 설정
4. **네트워크 구성**: `sc_service::build_network` 로 P2P 네트워크 설정
5. **RPC 서비스 구성**: 클라이언트와의 통신을 위한 RPC 인터페이스 설정
6. **컨센서스 메커니즘 시작**: 밸리데이터 모드에서 AURA 컨센서스 시작

부분 컴포넌트 생성 (`new_partial`)

```
pub fn new_partial<RuntimeApi, Executor>(
config: &Configuration,
dalek_end_block: Option<u32>,
) -> Result<PartialComponents<...>, sc_service::Error>
```

이 함수는 다음 컴포넌트들을 생성합니다:

- **텔레메트리 워커**: 노드 상태 모니터링을 위한 텔레메트리 시스템
- **실행자(Executor)**: WASM 런타임 실행을 위한 `NativeElseWasmExecutor`
- **클라이언트**: 블록체인 상태 관리를 위한 풀 클라이언트
- **백엔드**: 데이터베이스 및 스토리지 백엔드
- **트랜잭션 풀**: 트랜잭션 대기열 관리
- **임포트 큐**: 블록 임포트 처리를 위한 큐

런타임별 실행자 구성

프로젝트는 두 개의 런타임을 지원하며, 각각 별도의 실행자를 가집니다:

```
pub struct PicassoExecutor;
impl sc_executor::NativeExecutionDispatch for PicassoExecutor {
```

```
type ExtendHostFunctions = frame_benchmarking::benchmarking::HostFunctions;

fn dispatch(method: &str, data: &[u8]) -> Option<Vec<u8>> {

picasso_runtime::api::dispatch(method, data)

}

fn native_version() -> sc_executor::NativeVersion {

picasso_runtime::version::native_version()

}

}



pub struct ComposableExecutor;

impl sc_executor::NativeExecutionDispatch for ComposableExecutor {

// Composable 런타임용 구현

}
```

6.1.2 클라이언트 구성 요소 분석

클라이언트 추상화 (`client.rs`)

노드는 다중 런타임을 지원하기 위해 클라이언트 추상화를 사용합니다:

```
#[derive(Clone)]

pub enum Client {

Picasso(Arc<PicassoClient>),

Composable(Arc<ComposableClient>),
```

```
}
```

이 열거형은 런타임에 따라 적절한 클라이언트를 선택할 수 있게 해주며, 매크로를 통해 공통 인터페이스를 제공합니다:

```
macro_rules!  match_client {
($self:ident, $method:ident($($param:ident),*)) => {
match $self {
Self::Picasso(client) => client.$method($($param),*),
Self::Composable(client) => client.$method($($param),*),
}
};
}
```

RPC 서비스 구성 (`rpc.rs`)

RPC 서비스는 외부 클라이언트와의 통신을 담당합니다:

```
pub  struct FullDeps<Client, Pool> {
pub  client: Arc<Client>,
pub  pool: Arc<Pool>,
pub  deny_unsafe: DenyUnsafe,
pub  chain_props: sc_chain_spec::Properties,
}
```

```
pub fn create<RuntimeApi, Executor>(
deps: FullDeps<...>,
) -> Result<jsonrpsee::RpcModule<()>, jsonrpsee::core::Error>
```

RPC 모듈은 다음과 같은 API들을 제공합니다:

- **System API**: 기본 시스템 정보 및 상태 조회
- **TransactionPayment API**: 트랜잭션 수수료 계산
- **Assets API**: 자산 관련 기능
- **Pablo API**: DEX 관련 기능
- **Farming API**: 파밍 관련 기능
- **Lending API**: 대출 관련 기능
- **CosmWasm API**: CosmWasm 스마트 컨트랙트 지원
- **IBC API**: IBC 프로토콜 지원

데이터베이스 및 스토리지

노드는 다음과 같은 스토리지 컴포넌트를 사용합니다:

- **FullBackend**: `TFullBackend<Block>` 로 정의된 풀 백엔드
- **상태 백엔드**: `StateBackendFor<FullBackend, OpaqueBlock>`
- **트랜잭션 풀**: `FullPool<OpaqueBlock, FullClient<RuntimeApi, Executor>>`

6.1.3 파라체인 콜레이터와 릴레이체인 연동 메커니즘

릴레이체인 인터페이스 구축

파라체인은 릴레이체인과의 통신을 위해 두 가지 방식을 지원합니다:

```
async fn build_relay_chain_interface(
polkadot_config: Configuration,
parachain_config: &Configuration,
telemetry_worker_handle: Option<TelemetryWorkerHandle>,
task_manager: &mut TaskManager,
collator_options: CollatorOptions,
) -> RelayChainResult<(Arc<(dyn RelayChainInterface + 'static)>, Option<Collator
```

1. **최소 릴레이체인 노드**: RPC를 통한 외부 릴레이체인 노드 연결
2. **인프로세스 릴레이체인**: 동일 프로세스 내에서 릴레이체인 노드 실행

파라체인 블록 임포트

파라체인 블록 임포트는 `ParachainBlockImport` 를 통해 처리됩니다:

```
pub fn parachain_build_import_queue<RuntimeApi, Executor>(
backend: Arc<FullBackend>,
client: Arc<FullClient<RuntimeApi, Executor>>,
config: &Configuration,
telemetry: Option<TelemetryHandle>,
task_manager: &TaskManager,
) -> Result<(
sc_consensus::DefaultImportQueue<OpaqueBlock, FullClient<RuntimeApi, Executor>>,
ParachainBlockImport<OpaqueBlock, Arc<FullClient<RuntimeApi, Executor>>, FullBac
```

```
), sc_service::Error>
```

콜레이터 모드 vs 풀노드 모드

노드는 설정에 따라 두 가지 모드로 실행됩니다:

콜레이터 모드 (Validator)

```
if  validator {
let  params = StartCollatorParams {
para_id,
relay_chain_interface: relay_chain_interface.clone(),
block_status: client.clone(),
announce_block,
client: client.clone(),
task_manager: &mut  task_manager,
spawner,
parachain_consensus,
import_queue: import_queue_service,
collator_key: collator_key.expect("Command line arguments do not allow this. qed
relay_chain_slot_duration,
recovery_handle: Box::new(overseer_handle),
sync_service,
};
start_collator(params).await?;
```

```
}
```

풀노드 모드

```
else {
let params = StartFullNodeParams {
client: client.clone(),
announce_block,
task_manager: &mut task_manager,
para_id,
relay_chain_interface,
relay_chain_slot_duration,
import_queue: import_queue_service,
recovery_handle: Box::new(overseer_handle),
sync_service,
};
start_full_node(params)?;
}
```

워프 싱크 메커니즘

노드는 빠른 동기화를 위해 워프 싱크를 지원합니다:

```
let warp_sync_params = match parachain_config.network.sync_mode {
```

```
sc_network::config::SyncMode::Warp => {
let target_block = warp_sync_get::<OpaqueBlock, _>(
para_id,
relay_chain_interface.clone(),
spawn_handle.clone(),
);
Some(WarpSyncParams::WaitForTarget(target_block))
},
_ => None,
};
```

워프 싱크는 릴레이체인의 최종화된 블록을 기준으로 파라체인의 타겟 블록을 결정합니다:

```
async fn wait_for_target_block<B, RCInterface>(
sender: oneshot::Sender<<B as BlockT>::Header>,
para_id: ParaId,
relay_chain_interface: RCInterface,
) -> Result<(), Box<dyn std::error::Error + Send + Sync>>
```

6.1.4 오프체인 워커 지원

노드는 오프체인 작업을 위한 워커를 지원합니다:

```
if parachain_config.offchain_worker.enabled {
```

```
sc_service::build_offchain_workers(
&parachain_config,
task_manager.spawn_handle(),
client.clone(),
network.clone(),
);
}
```

이는 Oracle 팰릿과 같이 외부 데이터를 가져와야 하는 기능들에 필수적입니다.

6.1.5 텔레메트리 시스템

노드는 모니터링을 위한 텔레메트리 시스템을 포함합니다:

```
let telemetry = config
.telemetry_endpoints
.clone()
.filter(|x| !x.is_empty())
.map(|endpoints| -> Result<_, sc_telemetry::Error> {
let worker = TelemetryWorker::new(16)?;
let telemetry = worker.handle().new_telemetry(endpoints);
Ok((worker, telemetry))
})
.transpose()?;
```

텔레메트리는 노드의 성능, 상태, 네트워크 정보 등을 외부 모니터링 시스템으로 전송합니다.

결론

Composable Finance의 노드 아키텍처는 Substrate의 모듈형 설계를 활용하여 확장 가능하고 유연한 파라체인 노드를 구현했습니다. 다중 런타임 지원, 효율적인 릴레이체인 연동, 그리고 포괄적인 RPC API를 통해 복잡한 DeFi 기능들을 안정적으로 제공할 수 있는 기반을 마련했습니다.